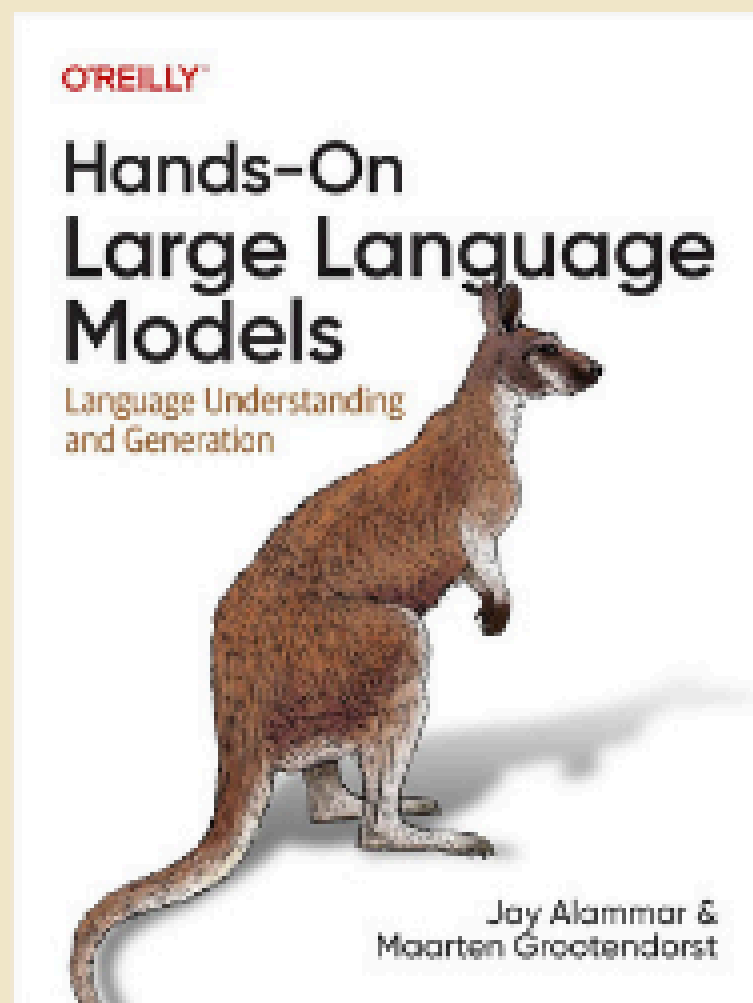
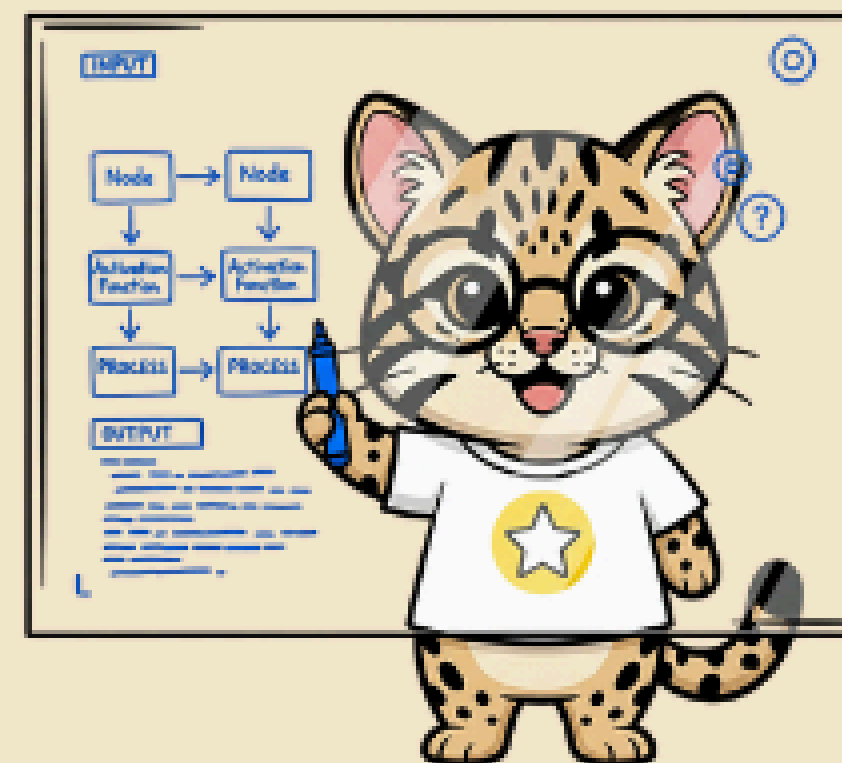




## 熬夜書坊



## Chapter 11

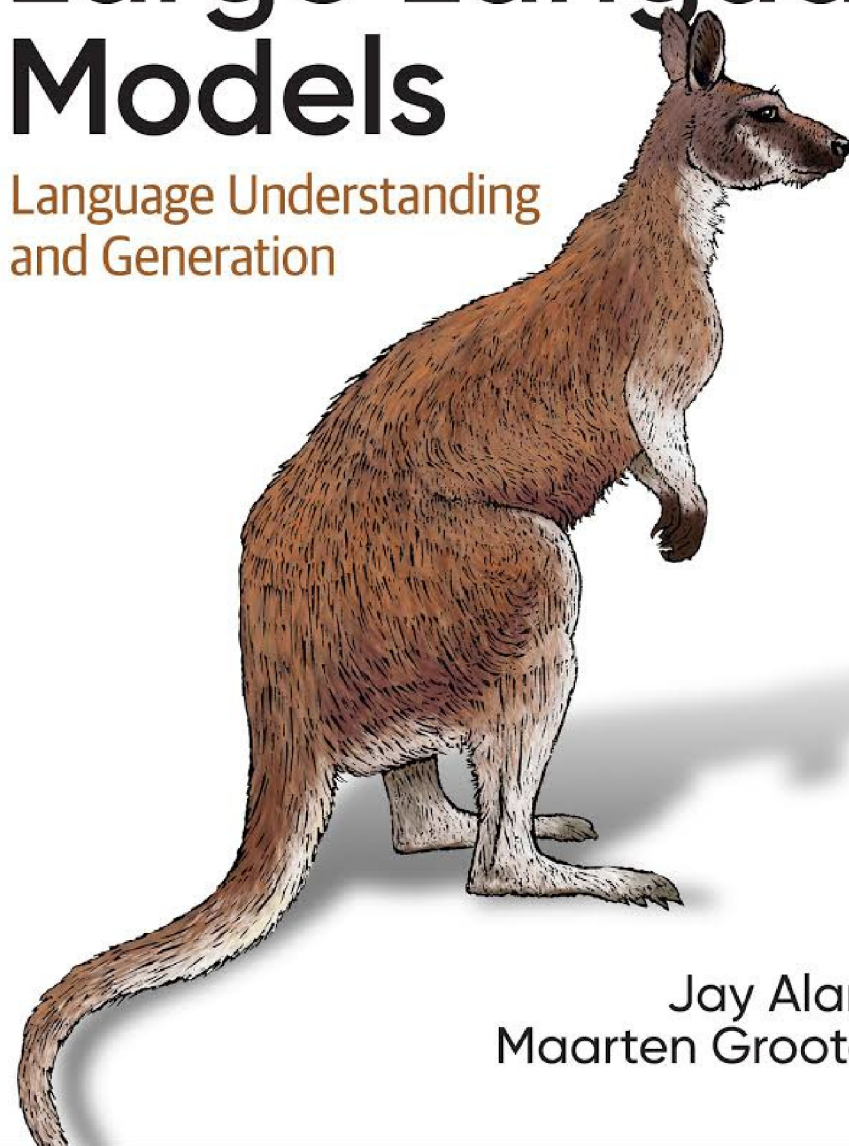


6/28 週日 21:00

O'REILLY®

# Hands-On Large Language Models

Language Understanding  
and Generation



Jay Alammar &  
Maarten Grootendorst

O'REILLY

# Hands-On Large Language Models

Language Understanding  
and Generation

---

Jay Alammar & Maarten Grootendorst

★ Twinkle AI 熬夜書坊 • Chapter 11

CHAPTER 11

# Fine-Tuning Representation Models for Classification

表示型模型的微調與分類任務

# 今日大綱

## Chapter 11 Overview

### 01 監督式分類

全參數微調 BERT，解凍層策略比較

### 02 少樣本分類

SetFit 框架：用 32 筆資料打出  $F1 = 0.85$

### 03 繼續預訓練

MLM 繼續訓練讓模型學會領域術語

### 04 命名實體識別

Token 級別分類：人名、組織、地點

# Ch.4 vs Ch.11：從 Frozen 到 Trainable

回顧對比

## Chapter 4：凍結模型

- 模型保持凍結 (Frozen)
- 嵌入模型輸出 → 分類頭
- 分類頭單獨訓練
- 快速、資源需求低

F1  $\approx$  **0.80**

## Chapter 11：全面微調

- ✓ BERT 模型可訓練
- ✓ BERT + 分類頭聯合更新
- ✓ Backward pass 穿透整個模型
- ✓ 表示能力更強、更精準

F1  $\approx$  **0.85** ↑



# Task-Specific Model 架構

BERT + Classification Head



關鍵差異：Backward pass 會穿透整個 BERT，所有 12 層的參數都會被更新

# 程式碼走讀：資料準備

Rotten Tomatoes Dataset + Tokenization

```
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from transformers import DataCollatorWithPadding

# 1. 載入 Rotten Tomatoes 資料集 (各 5,331 筆正負評)
tomatoes = load_dataset("rotten_tomatoes")
train_data, test_data = tomatoes["train"], tomatoes["test"]

# 2. 載入模型與 Tokenizer
model_id = "bert-base-cased"
model = AutoModelForSequenceClassification.from_pretrained(model_id, num_labels=2)
tokenizer = AutoTokenizer.from_pretrained(model_id)

# 3. Tokenize: 動態 padding 到 batch 內最長序列
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

# 程式碼走讀：訓練與評估

TrainingArguments & Trainer

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    load_f1 = evaluate.load("f1")
    return {"f1": load_f1.compute(predictions=predictions, references=labels)["f1"]}

training_args = TrainingArguments(
    "model",
    learning_rate=2e-5,           # BERT 微調經典學習率
    per_device_train_batch_size=16,
    num_train_epochs=1,
    weight_decay=0.01,
)
trainer = Trainer(
    model=model, args=training_args,
    train_dataset=tokenized_train, eval_dataset=tokenized_test,
```



## 結果對比：F1 Score

Ch.4 — 凍結模型

0.80

只訓練分類頭



Ch.11 — 全參數微調

0.85

所有層聯合訓練



# Layer Freezing 策略

凍結多少層最合適？

## 全凍結

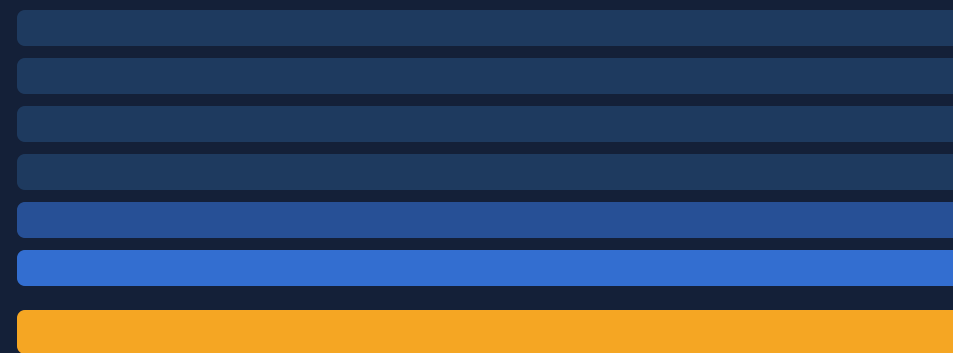
(只訓練分類頭)



F1 = 0.63

## 凍結前 10 層

(最後 2 層 + 分類頭)



F1 = 0.80

## 全解凍

(所有 12 層 + 分類頭)



F1 = 0.85

結論：只需訓練後半段 encoder blocks 就能接近全微調效果！

# Layer Freezing：程式碼實作

```
param.requires_grad = False
```

```
# — 策略 A：只解凍分類頭 ————— F1 = 0.63
for name, param in model.named_parameters():
    if name.startswith("classifier"):
        param.requires_grad = True    # 分類頭可訓練
    else:
        param.requires_grad = False   # BERT 全凍結
# — 策略 B：解凍最後 2 個 Encoder Block ————— F1 = 0.80
# index 165 之後的 block 需要解凍
if index < 165:
    param.requires_grad = False       # 前 10 個 block 凍結
# — 策略 C：完整微調（預設行為） ————— F1 = 0.85
# 所有參數都可訓練
```

## 解凍層數 vs. F1 Score 趨勢

前 5 層效果提升最顯著，之後趨緩



## SECTION 02

# 少樣本分類

## Few-Shot Classification

---

SetFit：只用 **32** 筆資料，打出  $F1 = 0.85$



# 少樣本分類：問題情境

標記資料稀少時怎麼辦？

## 現實挑戰

- ✗ 標記資料非常昂貴
- ✗ 領域專家時間有限
- ✗ 每個類別只有少量樣本
- ✗ 傳統微調需要大量資料才能避免過擬合
- ✗ 新任務冷啟動問題

## SetFit 解方

- ✓ 每個類別只需 8~16 筆
- ✓ 利用預訓練知識避免過擬合
- ✓ 透過對比學習增強訓練信號
- ✓ 無需 Prompt Engineering
- ★  **$F1 \approx 0.85$  (總共 32 筆!)**

# SetFit 三步驟演算法

Efficient Few-Shot Learning without Prompts

1

## 生成句子對

依類別內 / 跨類別選取  
正例（相似）和負例（不相似）句  
子對



2

## 微調嵌入模型

用對比學習（Contrastive  
Learning）  
在句子對上微調  
SentenceTransformer



3

## 訓練分類器

用微調後的嵌入生成特徵  
訓練邏輯迴歸（Logistic  
Regression）

# Step 1：生成正例 / 負例子對

In-class & Out-class Selection

| 原始標記資料（4 筆示例） |      | 生成的句子對        |          |      |
|---------------|------|---------------|----------|------|
| 文字            | 類別   | 文字 1          | 文字 2     | 類型   |
| 我在 Python 寫程式 | 程式語言 | 我在 Python 寫程式 | 我應該練 SQL | ✓ 正例 |
| 我應該練 SQL      | 程式語言 | 我的狗是黃金獵犬      | 我養了一隻暹羅貓 | ✓ 正例 |
| 我的狗是黃金獵犬      | 寵物   | 我在 Python 寫程式 | 我的狗是黃金獵犬 | ✗ 負例 |
| 我養了一隻暹羅貓      | 寵物   | 我養了一隻暹羅貓      | 我應該練 SQL | ✗ 負例 |



## Step 2：對比學習微調嵌入模型

Contrastive Learning with Sentence Transformers

### 正例對（同類別）

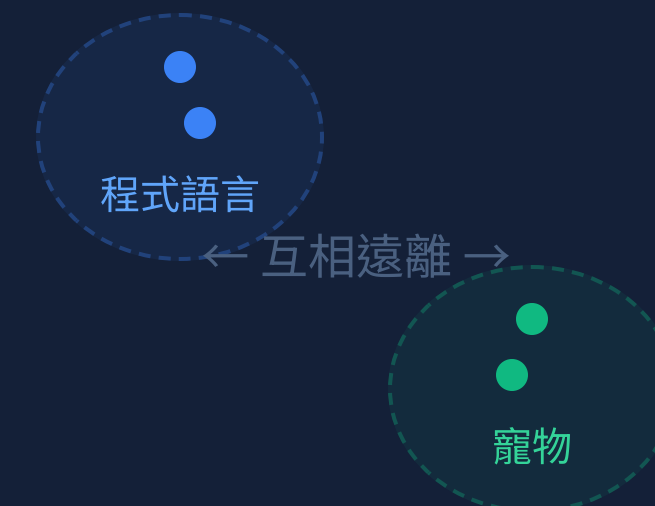
→ 讓這兩個 embedding  
在向量空間中**靠近**

### 負例對（不同類別）

→ 讓這兩個 embedding  
在向量空間中**遠離**

結果：模型學會「語意相似 = 類別相同」的表示

向量空間示意



## Step 3：訓練分類器（程式碼）

Embed → Classify

```
from setfit import SetFitModel, SetFitTrainer, sample_dataset

# 每類取 16 筆 → 共 32 筆
sampled_train = sample_dataset(tomatoes["train"], num_samples=16)

# 載入 SentenceTransformer 基礎模型
model = SetFitModel.from_pretrained("sentence-transformers/all-mpnet-base-v2")

args = SetFitTrainingArguments(
    num_epochs=3,          # 對比學習訓練輪數
    num_iterations=20      # 每個樣本產生的句子對數 (20×32×2 = 1,280 對)
)
trainer = SetFitTrainer(
    model=model, args=args,
    train_dataset=sampled_train,
    eval_dataset=test_data, metric="f1",
```

## SetFit 效果驚人：資料量 vs. F1

全量微調 (Ch.11)

8,500

筆標記資料

F1 = 0.85

VS

資料量差  
265×

F1 差距  
< 1%

SetFit (Few-Shot)

32

筆標記資料

F1 = 0.84 ★

## SECTION 03

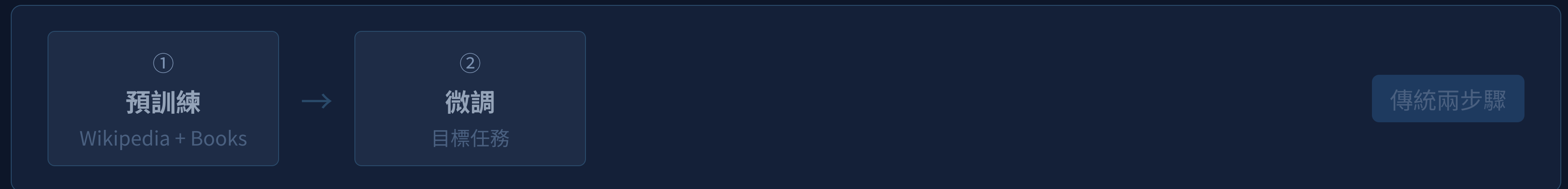
# 繼續預訓練

Continued Pretraining  
with Masked Language Modeling

三步驟流程：預訓練 → 繼續預訓練 → 微調

# 兩步驟 vs. 三步驟訓練流程

加入 Continued Pretraining 的價值



例如：通用 BERT → BioBERT（醫療領域繼續預訓練）→ 藥物分類微調



## 繼續預訓練：程式碼實作

AutoModelForMaskedLM + DataCollatorForLanguageModeling

```
# 1. 載入 MLM 模型 (不是分類模型!)
model = AutoModelForMaskedLM.from_pretrained("bert-base-cased")
tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")

# 2. 移除分類標籤 (MLM 是自監督任務)
tokenized_train = train_data.map(preprocess_function, batched=True)
tokenized_train = tokenized_train.remove_columns("label") # ← 移除!
# 3. Token Masking: 隨機遮蔽 15% 的 Token
data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer, mlm=True, mlm_probability=0.15
)

# 4. 訓練並儲存繼續預訓練後的模型
trainer = Trainer(model=model, args=training_args,
                  train_dataset=tokenized_train,
                  data_collator=data_collator)

trainer.train()
```

## 繼續預訓練效果驗證

Mask Filling：領域知識植入成功！

### 原始 BERT

What a horrible [MASK]!

1. What a horrible **idea**!
2. What a horrible **dream**!
3. What a horrible **thing**!
4. What a horrible **day**!
5. What a horrible **thought**!

→ 通用語言知識，沒有電影感

### 繼續預訓練後

What a horrible [MASK]!

1. What a horrible **movie**! ★
2. What a horrible **film**! ★
3. What a horrible **mess**!
4. What a horrible **comedy**!
5. What a horrible **story**!

→ 電影術語顯著提升！

## SECTION 04

# 命名實體識別

## Named-Entity Recognition

---

從文件分類到 Token 分類



# 文件分類 vs. Token 分類

NER 的核心架構差異

## 文件分類 (Ch11 前三節)

- 輸入：整篇文章
- 取 [CLS] token embedding
- 分類頭：1 次預測
- 輸出：文件標籤

[CLS] This movie is great

Positive

## Token 分類 (NER)

- 輸入：一個句子
- 取每個 token 的 embedding
- 分類頭：每個 token 都預測一次
- 輸出：每個 token 的實體標籤

Dean Palmer hit Rangers  
B-PER I-PER O B-LOC

# BIO 標記方案：B / I / O

處理多 Token 實體的標準方法

## B-XXX

Beginning：某個實體類型的開始

B-PER = 人名開始

## I-XXX

Inside：屬於前一個實體的延續

I-PER = 人名延續

## O

Outside：不屬於任何命名實體

普通詞彙、動詞等

範例句子

Dean

B-PER

Palmer

I-PER

hit

O

his

O

30th

O

homer

O

for the

O

Rangers

B-LOC

# Subword 標籤對齊挑戰

一個詞被切成多個 Token 怎麼辦？

問題：BERT 的 Tokenizer 會切割詞語

|            |    |      |    |         |        |       |
|------------|----|------|----|---------|--------|-------|
| 輸入詞語       | My | name | is | Maarten |        |       |
| Tokenize 後 | My | name | is | Ma      | ##arte | ##n   |
| 對齊標籤       | O  | O    | O  | B-PER   | I-PER  | I-PER |

✓ 詞語第一個 subtoken → 保留原始標籤（B 或 O）

✓ 後續 subtokens → 改為 I（Inside）標籤

✓ [CLS]/[SEP] 特殊 token → -100（忽略）

# NER 程式碼：標籤對齊 align\_labels

WordPiece Subtoken 對齊核心邏輯

```
def align_labels(examples):
    token_ids = tokenizer(examples["tokens"],
                          truncation=True, is_split_into_words=True)

    labels = examples["ner_tags"]
    updated_labels = []
    for index, label in enumerate(labels):
        word_ids = token_ids.word_ids(batch_index=index)
        previous_word_idx, label_ids = None, []
        for word_idx in word_ids:
            if word_idx != previous_word_idx:      # 新詞語第一個 subtoken
                previous_word_idx = word_idx
                updated_label = -100 if word_idx is None else label[word_idx]
            elif word_idx is None:                  # [CLS]/[SEP] 特殊 token
                updated_label = -100
            else:                                   # 同一詞後續 subtokens
                updated_label = label[word_idx]
```

# NER 訓練 & 推論結果

AutoModelForTokenClassification

```
from transformers import (
    AutoModelForTokenClassification,
    pipeline
)
# 載入 Token Classification 模型
model = AutoModelForTokenClassification.from_pretrained(
    "bert-base-cased",
    num_labels=len(id2label),
    id2label=id2label, label2id=label2id
)
data_collator = DataCollatorForTokenClassification(
    tokenizer=tokenizer
)
# ... 訓練過程與文件分類相同 ...
trainer.save_model("ner_model")

# 推論
```

## 推論結果

| entity | score | word   |
|--------|-------|--------|
| B-PER  | 0.995 | Ma     |
| I-PER  | 0.993 | ##arte |
| I-PER  | 0.995 | ##n    |



# 本章四大方法綜合比較

How to Choose?

| 方法         | 資料需求          | 訓練時間        | F1     | 適用場景    |
|------------|---------------|-------------|--------|---------|
| 全參數微調      | 大量 (~8,500筆)  | 快 (1 epoch) | 0.85 ★ | 有足夠標記資料 |
| 凍結部分層      | 大量            | 更快          | 0.80   | 計算資源有限  |
| SetFit     | 極少 (~32筆)     | 中等          | 0.84 ★ | 標記資料稀少  |
| 繼續預訓練 + 微調 | 大量無標籤 + 少量有標籤 | 最長          | 最好     | 領域特定術語多 |

# 本章重點回顧

## Chapter 11 Key Takeaways

**01** 全參數微調 BERT 比只訓練分類頭效果更好 (F1 **0.80** → **0.85**)，但需要更多計算資源

**02** 凍結策略：通常只需解凍後半段 encoder blocks 就能取得接近全微調的效果

**03** SetFit 是少樣本分類利器：**32 筆**資料靠對比學習擴增，打出 F1 = 0.84

**04** MLM 繼續預訓練讓模型吸收**領域術語**，是三步驟流程的關鍵中間層

**05** NER 是 Token 級別分類，需要 **BIO 標記方案**和 Subword 標籤對齊 (align\_labels)

# Q & A

有任何問題，歡迎舉手或在 Discord 發問！

下週預告 — Chapter 12

## Fine-Tuning Generative Models

生成式模型微調：SFT 監督式微調 + RLHF / DPO 人類偏好對齊

感謝參與 Twinkle AI 熬夜書坊

ai-twinkle/LLM-Book-Club • @\_thliang01